5️⃣

MATERI WEEK 05 - Synchronization

A. REVIEW SESI LECTURE

1. Konsep Dasar :

a. Synchronization

Synchronozation adalah teknik untuk mengatur akses beberapa process atau thread terhadap resource bersama agar tidak terjadi konflik. Hal ini disebabkan karena pada Sistem Operasi Modern, banyak Process berjalan secara Concurrent / Bersamaan. Hal ini akan menjadi masalah ketika mereka mengakses shared resource seperti:

- Variabel Global
- File
- Shared Memory, dll

b. Race Condition

Race Condition terjadi ketika hasil program bergantung pada urutan eksekusi thread atau process. Hal ini terjadi karena beberapa process berlomba-lomba untuk membaca dan menulis ke shared resource yang menyebabkan adanya Interleaving (Menimpa satu sama lain).

c. Critical Region / Critical Section

Critical Region adalah bagian dari program dimana sebuah process atau thread mengakses shared resources. Critical Region tidak boleh dijalankan oleh lebih dari 1 thread / process pada waktu yang sama.

- Analoginya : Mesin ATM (Hanya boleh 1 orang yang menggunakan ATM)

```
orang A tarik uang
orang B tarik uang

Hasilnya : Saldo bisa jadi salah / tidak sinkron

Solusinya : Hanya boleh 1 orang yang menggunakan ATM
(Mutual Exclusion)
```

2. Mutex Locks

Mutex Locks adalah solusi paling sederhana yang dibuat untuk menyelesaikan masalah Critical Section.

Cara kerjanya adalah dengan melinduingi Critical Section, dimana sebuah Process harus menggunakan fungsi `acquire()` terlebih dahulu unutk mengunci (Mendapatkan akses), kemudian memakai fungsi `release()` untuk melepaskannya setelah selesai.

Kedua fungsi `acquire()` dan `release()` ini wajib berjalan secara utuh tanpa bisa diinterupsi (Atomic).

Kekurangan dari implementasi dasar Mutex Lock adalah adanya kemungkinan terjadinya Busy Waiting, yaitu process akan terus mengulang loop tanpa henti hanya untuk menunggu kunci tersebut tersedia. Kunci dengan tipe ini sering disebut dengan spinlock.

3. Semaphores

Semaphore adalah alat sinkronisasi yang lebih baik dibandingkan Mutex Locks. Semaphore berbentuk sebuah variabel bilangan bulat (Integer) berinisial S yang hanya dapat diakses melalui 2 operasi atomic yaitu `wait()` dan `signal()` .

a. Jenis-Jenis Semaphore

Ada 2 jenis Semaphore yaitu:

1. Binary Semaphore, Nilainya hanya bisa 0 dan 1. Ini sangat mirip dengan Mutex dan sering digunakan untuk memastikan hanya 1 process yang mengakses sumber daya pada 1 waktu.
2. Counting Semaphore, Nilainya bisa lebih dari 1, Jenis ini digunakan untuk mengatur akses ke Resources yang memiliki lebih dari 1 instance (Contohnya mengatur akses ke 5 koneksi database sekaligus)

ANALOGI :

Bayangkan sebuah tempat parkir dengan 10 slot (Counting Semaphore = 10). Setiap kali sebuah mobil masuk, penjaga gerbang mengurangi angka (Wait) menjadi 9. Jika angka mencapai 0, mobil yang baru datang harus menunggu di luar. Saat ada mobil yang keluar, penjaga menaikkan angka (Signal) menjadi 1, sehingga mobil yang menunggu bisa masuk.

b. Perbedaan Antara Mutex dan Binary Semaphore

Perbedaan dari kedua konsep ini adalah mengenai Kepemilikan. Pada Mutex, hanya pemilik dari Mutex yang bisa melepas kuncinya, sedangkan pada Binary

Semaphore tidak memiliki aturan kepemilikan tersebut.

c. Semaphore Solusi dari Masalah Busy Waiting pada Mutex

Untuk mengatasi kekurangan pemborosan waktu akibat busy waiting, Semaphore diimplementasikan dengan antrean tunggu (Waiting Queue). Operasi `block()` akan menempatkan process yang sedang menungguy ke dalam antrean dan `wakeup()` untuk memindahkannya kembali ke antrean siap dieksekusi (Ready Queue).

4. Deadlock dan Starvation

- Let s and q be two semaphores initialized to 1

P_0	P_1
`wait(S);`	`wait(Q);`
`wait(Q);`	`wait(S);`
`...`	`...`
`signal(S);`	`signal(Q);`
`signal(Q);`	`signal(S);`

Semaphore ordering is essential!
Wrong ordering causes deadlock!

Deadlock adalah kondisi dimana dua atau lebih process saling menunggu tanpa batas waktu untuk menunggu sebuah peristiwa yang hanya bisa dipicu oleh salah satu dari Process yang saling menunggu tersebut. Kesalahan dalam mengurutkan semaphore dapat menyebabkan deadlock

Starvation adalah ketika terjadinya pemblokiran abadi. Ini terjadi ketika sebuah process bisa saja terjebak dan tidak pernah ditarik keluar dari antrean semaphore. (Gak pernah dapat makan wok)

Priority Inversion adalah suatu masalah penjadwalan ketika process dengan prioritas lebih rendah malah memegang kunci yang sangat dibutuhkan oleh process dengan prioritas lebih tinggi. Hal ini dapat diatasi melalui priority-interitance protocol.

5. Classical Problem

1. Producer-Consumer Problem (Bounded-Buffer Problem)Masalah ini menggambarkan skenario sinkronisasi di mana terdapat proses yang memproduksi barang (produsen) dan proses yang mengonsumsinya (konsumen) menggunakan wadah penyangga (*buffer*) yang terbatas.

- **Karakteristik**: Terdapat *n* buah *buffer*, di mana setiap *buffer* hanya bisa menampung satu *item* saja.
- **Semaphore yang digunakan**:
  - `mutex` (diinisialisasi dengan nilai 1) untuk memberikan perlindungan *mutual exclusion* atau mengamankan *critical section*.
  - `full` (diinisialisasi dengan nilai 0) sebagai penanda atau *event semaphore* untuk menghitung jumlah slot yang sudah terisi.
  - `empty` (diinisialisasi dengan nilai *n*) sebagai penanda untuk menghitung jumlah slot yang masih kosong.
- **Siklus Produsen (Producer Process)**: Proses ini memanggil `wait(empty)` untuk menahan jika *buffer* penuh (menunggu slot kosong), lalu mengunci *buffer* dengan `wait(mutex)`. Setelah memproduksi dan menambahkan *item* ke dalam *buffer*, produsen akan melepaskan kunci tersebut dengan `signal(mutex)` dan menambah jumlah slot yang terisi melalui `signal(full)`.
- **Siklus Konsumen (Consumer Process)**: Proses ini memanggil `wait(full)` untuk menahan jika *buffer* kosong (menunggu slot terisi), lalu mengunci *buffer* dengan `wait(mutex)`. Setelah membuang/mengambil *item* dari *buffer*, konsumen akan melepaskan kunci perlindungan dengan `signal(mutex)` dan menginformasikan penambahan jumlah slot yang kosong lewat `signal(empty)`.

2. Readers-Writers Problem

Masalah ini difokuskan pada pengaturan satu kelompok data (*data set*) yang dibagikan secara bersama-sama kepada sejumlah proses yang berjalan dengan konkuren (serentak).

- **Karakteristik Proses**:
  - **Readers (Pembaca)**: Hanya diperbolehkan membaca set data tersebut dan tidak diizinkan melakukan pembaruan/modifikasi.
  - **Writers (Penulis)**: Memiliki akses untuk membaca sekaligus menulis atau mengubah data.
  - Aturan utamanya adalah sistem harus memperbolehkan beberapa pembaca (*multiple readers*) untuk membaca data pada saat yang

bersamaan. Namun, untuk menghindari inkonsistensi, ketika ada satu *writer* tunggal yang sedang mengakses data, maka proses lain (baik pembaca maupun penulis lain) tidak boleh mengakses data tersebut.

- **Semaphore & Variabel yang digunakan**:
  - Semaphore `rw_mutex` (diinisialisasi dengan nilai 1) digunakan untuk mengunci data.
  - Semaphore `mutex` (diinisialisasi dengan nilai 1) digunakan untuk mengamankan variabel perhitungan.
  - Variabel angka bulat (integer) `read_count` (diinisialisasi dengan nilai 0) untuk melacak jumlah *reader* yang sedang membaca data.
- **Cara Kerja Writer**: Proses penulis akan mengunci data melalui `wait(rw_mutex)`, lalu setelah selesai menulis, ia melepaskannya dengan `signal(rw_mutex)`.
- **Cara Kerja Reader**: Ketika *reader* akan masuk, ia melakukan perhitungan `read_count++` secara aman dengan perlindungan `mutex`. Jika ia adalah orang pertama yang membaca data tersebut (`read_count == 1`), maka ia akan mengunci `rw_mutex` agar proses *writer* tidak bisa masuk mengganggu. Setelah selesai, *reader* kembali menggunakan `mutex` untuk mengurangi jumlah pembaca (`read count--`); jika ia adalah pembaca terakhir yang keluar (`read_count == 0`), ia akan melepaskan kunci `signal(rw_mutex)` sehingga penulis akhirnya bisa masuk kembali.
- **Variasi & Masalah Tambahan**: Ada variasi dari masalah ini (misalnya *writer* langsung mendapat prioritas utama saat ia sudah siap menulis), dan semua bentuk variasi melibatkan beberapa bentuk penjadwalan prioritas. Sayangnya, hal ini bisa berujung pada *starvation* (pemblokiran abadi pada proses yang tidak kunjung mendapat giliran). Di sistem tertentu, masalah ini dapat diatasi melalui kernel yang menyediakan fasilitas *reader-writer locks* bawaan.

3. Dining-Philosophers Problem Masalah sinkronisasi klasik ini merepresentasikan kasus pengalokasian sumber daya tunggal kepada proses-proses yang membutuhkannya secara adil.

- **Karakteristik**: Diibaratkan ada kumpulan filsuf (*philosophers*) di meja bundar yang hanya melakukan dua aktivitas terus-menerus: berpikir (*think*) dan makan (*eat*) secara bergantian.

- **Algoritma dengan Sumpit (Chopstick)**:
  - Misalnya terdapat filsuf yang diidentifikasi dengan indeks *i*. Agar filsuf tersebut bisa makan, ia harus mengambil dua buah sumpit sekaligus di sisi kiri dan kanannya.
  - Ia akan mengambil sumpit sebelah kirinya dengan mengeksekusi `wait(chopstick[i])`, lalu disusul mengambil sumpit sebelah kanannya dengan mengeksekusi `wait(chopStick[(i + 1) % 5])`.
  - Setelah berhasil memegang kedua sumpit, barulah filsuf tersebut bisa **makan (eat)**.
  - Setelah selesai makan, filsuf akan meletakkan kembali sumpit kirinya lewat instruksi `signal(chopstick[i])` dan meletakkan sumpit kanannya dengan `signal(chopstick[(i + 1) % 5])`, lalu kembali **berpikir (think)**

Dalam Dining Philosophers Problem, ada masalah yang mungkin terjadi yaitu deadlock. Sebuah kondisi dimana jika seluruh filsuf tiba-tiba mengambil sumpit kiri secara bersamaan, dimana mereka semua akan saling tertahan selamanya karena tidak akan pernah ada sumpit kanan.

6. Monitors

- *Monitors* adalah tipe data abstrak dan tingkat tinggi yang memberikan mekanisme mudah dan efektif untuk sinkronisasi antaran proses.
- Pada *Monitors*, hanya diizinkan satu proses saja yang boleh aktif di dalam monitor tersebut pada satu waktu. Variabel di dalamnya hanya bisa diakses oleh kode-kode yang ada dalam prosedur monitor tersebut.
- *Monitors* memiliki *Condition Variables* (misalnya `x`, `y`) yang memiliki perintah `x.wait()` (proses yang memanggil akan ditunda sementara) dan `x.signal()` (melanjutkan eksekusi proses yang tadi ditunda).
- Saat beberapa proses mengantre dan `signal()` dijalankan, proses yang akan dilanjutkan bisa disesuaikan dengan penjadwalan prioritas berdasarkan perintah `x.wait(c)`, di mana *c* merupakan angka prioritas dari proses tersebut.
- *Monitors* bisa dialokasikan untuk membagikan satu buah sumber daya kepada proses-proses yang bersaing secara adil sesuai kebutuhan durasi maksimal

mereka memakai fasilitasnya.

B. GUIDED PRACTICUM

1. PRACTICUM 5.1 : Unnamed Semaphore

Semaphore pada dasarnya adalah penghitung (Counter) untuk shared resources yang dipakai bersamaan oleh Process atau Thread. Operasi dasar pada Semaphore adalah menaikkan nilai penghitung secara atomik, dan menunggu sampai nilai penghitung tidak bernilai Nol. Kemudian menurunkannya secara atomic.

Atomic Operation adalah sebuah Operasi atau serangkaian instruksi yang dieksekusi sebagai satu kesatuan utuh yang tidak dapat dibagi, diinterupsi, dan dihentikan di tengah jalan. Prinsip utama Atomic adalah All-Or-Nothing (semua atau tidak sama sekali) artinya ketika sebuah sistem mengeksekusi operasi atomik, operasinya akan selesai secara keseluruhan dengan sukses. Tidak akan pernah ada keadaan dimana operasi tersebut hanya selesai setengahnya.

Pada praktikum ini, kita akan melakukan eksperimen menggunakan **POSIX semaphore** untuk sinkronisasi, termasuk **koordinasi sinyal (signaling coordination)** dan **mutual exclusion**. Beberapa operasi dasar semaphore yang diperlukan adalah: **pembuatan dan inisialisasi**, **down/decrement (penurunan nilai)**, **up/increment (peningkatan nilai)**, dan **penghancuran (destruction)**.

The POSIX standard mendefinisikan dua jenis semaphore yaitu Unnamed Semaphore dan Named Semaphore. Ada beberapa hal yang perlu diperhatikan ketika menggunakan Unnamed Semaphore:

```
// File Header yang digunakan
#include <semaphore.h>


// Tipe Data Semaphore
sem_t
```

```
// CONTOH:
sem_t sem1; // Artinya membuat variabel semaphore dengan nama
sem1


// Inisialisasi Semaphore
int sem_init(sem_t *sem, int pshared, unsigned value);
// PARAMETER:
sem_t sem1;
sem_init(&sem1, 0, 1);
// PENJELASAN:
1) sem_t *sem => Pointer ke semaphore yang akan dinisialisai
2) int pshared => Menentukan apakah semaphore digunakan,
misal: 0 artinya digunakan antar thread dalam 1 process
             1 artinya digunakan antar process berbeda
3) unsigned value => Nilai awal semaphore
misal: value = 1 artinya hanya 1 thread yang boleh masuk ke c
ritical section
// Value pada Binary Semaphore
1) Binary Semaphore, value = 1 artinya digunakan untuk mutual
exclusion
2) Counting Semaphore, value > 1 artinya digunakan untuk meng
ontrol banyak resources


// Operasi Semaphore yang Tersedia
int sem_wait(sem_t *sem);   // operasi down / P / wait Operat
ion semaphore => Mengurangi Nilai Semaphore
int sem_post(sem_t *sem);   // operasi up / V / signal Operat
ion semaphore => Menaikkan Nilai Semaphore
int sem_destroy(sem_t *sem); // Membebaskan  / Menghancurkan
Semaphore yang tidak digunakan lagi
```

```
// Cara Melakukan Kompilasi Program
gcc -o filename filename.c -pthread
```

a. praksem-1_NIM.c

Tulis program dibawah, compile, dan jalankan. Jelaskan apa yang terjadi dan observasi Process status dengan perintah `ps axj | grep praksem-1`

```
// SAVE DENGAN NAMA : praksem-1_NIM.c
#include <semaphore.h>
#include <st••••••••dio.h>

// Nilai awal Semaphore
#define INITVALUE 1
// Loop dijalankan 10 kali
#define MAXCOUNT 10

int main() {
    sem_t semid;
    int i, sem_value;

    // Inisialisai Semaphore
    if (sem_init(&semid, 0, INITVALUE) == -1) {
    // Kalau inisialisasi gagal maka error
        perror("Failed to initialize semaphore\n");
        return 1;
    }
    // Loop masuk 10 kali
    for (i = 0; i < MAXCOUNT; i++) {
        // Mengambil nilai semaphore saat ini dan disimpan ke
sem_value
        if (sem_getvalue(&semid, &sem_value) == -1) {
            perror("Failed to get semaphore value\n");
        }
        // Menampilkan nilai semaphore
        printf("Semaphore_value = %d \n", sem_value);
```

```
        // Bagian ini melakukan semaphore = semaphore - 1
        if (sem_wait(&semid) == -1) { // Down semaphore, atom
ically
            perror("Failed to decrement semaphore\n");
        }
    }
}
```

```
vboxuser@Ubuntu:~/Desktop/week05$ nano praksem1.c
vboxuser@Ubuntu:~/Desktop/week05$ gcc praksem1.c -o praksem1 -pthread
vboxuser@Ubuntu:~/Desktop/week05$ ./praksem1
Semaphore_value = 1
Semaphore_value = 0
```

```
vboxuser@Ubuntu:~/Desktop/week05$ ps axj | grep praksem1
   2856    4691    4691    2856 pts/0       4691 S+    1000   0:00 ./praksem1
   4736    4788    4787    4736 pts/1       4787 S+    1000   0:00 grep --color=auto praksem1
vboxuser@Ubuntu:~/Desktop/week05$
```

HASILNYA: Process berada di Sleeping State, hal ini terjadi karena process sedang menunggu sesuatu (Blocked).

Modifikasi Program diatas dengan menambahkan **sem_post(&semid) operation to increment the semaphore value after sem_wait(&semid). What happens? SAVE Dengan Nama : praksem-1-mod_NIM.c**

```
// SAVE DENGAN NAMA : praksem-1-mod_NIM.c
#include <semaphore.h>
#include <stdio.h>

#define INITVALUE 1
#define MAXCOUNT 10

int main()
{
    sem_t semid;
    int i, sem_value;
```

```
    if (sem_init(&semid, 0, INITVALUE) == -1) {
        perror("Failed to initialize semaphore");
        return 1;
    }

    for(i = 0; i < MAXCOUNT; i++) {

        if (sem_getvalue(&semid, &sem_value) == -1)
            perror("Failed to get semaphore value");

        printf("Semaphore_value = %d\n", sem_value);

        if (sem_wait(&semid) == -1) // down semaphore
            perror("Failed to decrement semaphore");

        /* Tambahan sesuai instruksi */
        if (sem_post(&semid) == -1) // up semaphore
            perror("Failed to increment semaphore");
    }

    sem_destroy(&semid);

    return 0;
}
```

HASILNYA

```
vboxuser@Ubuntu:~/Desktop/week05$ gcc praksem-1-mod.c -o praksem-1-mod -pthread
vboxuser@Ubuntu:~/Desktop/week05$ ./praksem-1-mod
Semaphore_value = 1
Semaphore_value = 1
Semaphore_value = 1
Semaphore_value = 1
Semaphore_value = 1
Semaphore_value = 1
Semaphore_value = 1
Semaphore_value = 1
Semaphore_value = 1
Semaphore_value = 1
vboxuser@Ubuntu:~/Desktop/week05$
```

```c
// EKPERIMEN.c

#include <semaphore.h>
#include <stdio.h>

#define INITVALUE 1
#define MAXCOUNT 10

int main()
{
    sem_t semid;
    int i, sem_value;

    if (sem_init(&semid, 0, INITVALUE) == -1) {
        perror("Failed to initialize semaphore");
        return 1;
    }

    for(i = 0; i < MAXCOUNT; i++) {

        /* menurunkan semaphore */
        if (sem_wait(&semid) == -1)
            perror("Failed to decrement semaphore");
```

```
        sem_getvalue(&semid, &sem_value);
        printf("Semaphore_value = %d\n", sem_value);

        /* menaikkan semaphore */
        if (sem_post(&semid) == -1)
            perror("Failed to increment semaphore");

        sem_getvalue(&semid, &sem_value);
        printf("Semaphore_value = %d\n", sem_value);
    }

    sem_destroy(&semid);
    return 0;
}
```

HASILNYA :

```
vboxuser@Ubuntu:~/Desktop/week05$ nano eksperimen.c
vboxuser@Ubuntu:~/Desktop/week05$ gcc eksperimen.c -o eksperimen -pthread
vboxuser@Ubuntu:~/Desktop/week05$ ./eksperimen
Semaphore_value = 0
Semaphore_value = 1
Semaphore_value = 0
Semaphore_value = 1
Semaphore_value = 0
Semaphore_value = 1
Semaphore_value = 0
Semaphore_value = 1
Semaphore_value = 0
Semaphore_value = 1
Semaphore_value = 0
Semaphore_value = 1
Semaphore_value = 0
Semaphore_value = 1
Semaphore_value = 0
Semaphore_value = 1
Semaphore_value = 0
Semaphore_value = 1
Semaphore_value = 0
Semaphore_value = 1
```

EFEK PERUBAHAN PADA PROGRAM :

1. Jika tidak ada `sem_post()` maka:
    - `sem_wait()` menurukan nilai semaphore
    - Nilai menjadi 0
    - Iterasi berikutnya program akan diblock
2. Jika ada `sem_post()` maka:
    - `sem_wait()` menurukan semaphore, semaphore = semaphore - 1
    - `sem_post()` langsung menaikkannya kembali
    - Nilai semaphore kembali ke 1
    - Loop dapat berjalan sampai 10 kali.

c. praksem-2_NIM.c

Tujuan program ini adalah menunjukkan:

- Sinkronisasi antar thread
- Bagaimana semaphore digunakan sebagai sinyal
- Bagaimana satu thread bisa menghentikan sementara thread lain dan melanjutkannya kembali.

```
Thread 1 → task1
Thread 2 → task2
```

```
// SAVE DENGAN NAMA : praksem-2_NIM.c
#include <stdlib.h> // Util Function
#include <unistd.h> // Sys Linux Function
#include <pthread.h> // Thread POSIX
#include <semaphore.h> // Semaphore POSIX
#include <stdio.h> // IO

// Loop akan berjalan sebanyak 500 kali
#define NLOOP 500
```

```
int g1 = 0; // Counter yang dihitung thread
sem_t semid1, semid2; // Semaphore untuk mengaktifkan task1 d
an task2
// Bisa jadi alat komunikasi antar thread


// Function yang dipanggil untuk menambah nilai g1
void task1(void *arg)
{
    int i;
    for(i = 0; i < NLOOP; i++) {
        g1++;
        printf("task1 count:%d g1:%d\n", i+1, g1);
                // Setiap kelipatan 5 akan berkomunikasi deng
an task2
        if (g1 % 5 == 0) {
                // Kirim Signal bahwa task2 boleh jalan
            sem_post(&semid2);
            // task1 berhenti sementara sampai task2 memberi
izin lanjut
            sem_wait(&semid1);
        }
    }
}

// Function yang dipanggil untuk menambah nilai g2
void task2(void *arg)
{
    while(1) {
            // task2 menunggu signal task1
        sem_wait(&semid2);

        if (g1 <= NLOOP) {
            printf(">>>>>>>>>>>>>>> task 2 g1: %d\n", g1);
            // Kirim signal bahwa task1 boleh jalan
```

```
            sem_post(&semid1);
                        // Kondisi berhenti, dimana jika task
2 dengan g1 = 500 maka akan berhenti
            if (g1 == NLOOP)
                break;
        }
    }
}

int main()
{
        // Buat Thread
    pthread_t thrd1, thrd2;

        // Inisialisasi Semaphore
    sem_init(&semid1, 0, 0);
    sem_init(&semid2, 0, 0);
    // Parameter 2: pshared, artinya semaphore digunakan anta
r thread dan bukan antar process
    // Parameter 3: Nilai awal semaphore = 0 artinya task2 ak
an menunggu sampai task1 memanggil

        // Membuat Thread
    pthread_create(&thrd1, NULL, (void *)task1, NULL);
    pthread_create(&thrd2, NULL, (void *)task2, NULL);

        // Menunggu thread selesai
    pthread_join(thrd2, NULL);
    pthread_join(thrd1, NULL);
}
```

QNA : Ada berapa thread ? dalam program ini ?

HASIL:

```
task1 count:486 g1:486
task1 count:487 g1:487
task1 count:488 g1:488
task1 count:489 g1:489
task1 count:490 g1:490
>>>>>>>>>>>>>>> task 2 g1: 490
task1 count:491 g1:491
task1 count:492 g1:492
task1 count:493 g1:493
task1 count:494 g1:494
task1 count:495 g1:495
>>>>>>>>>>>>>>> task 2 g1: 495
task1 count:496 g1:496
task1 count:497 g1:497
task1 count:498 g1:498
task1 count:499 g1:499
task1 count:500 g1:500
>>>>>>>>>>>>>>> task 2 g1: 500
```

c. praksem-3_NIM.c

Tujuan dari program ini adalah untuk:

- Menunjukkan sinkronisasi antar process
- Menggunakan shared memory
- Menggunakan semaphore yang ditempatkan di shared memory
- Memperlihatkan race condition ketika tidak ada mutual exclusion

Program ini berbeda dengan praksem-2 yang menggunakan thread, program ini menggunakan process.

```
// SAVE DENGAN NAMA :praksem-3_NIM.c
#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <semaphore.h>

#define key  ((key_t) 7890)
#define key2 ((key_t) 7891)

int main(int argc, char **argv)
{
        // Membuat beberapa variabel
    int i, nloop = 500000, zero = 0, value, *ptr;
    int shmid, semid, mode, shm_size, sem_size, status;
    sem_t *mutex;

     // Membuat shared memory
    shm_size = sizeof(int);
    shmid = shmget(key, shm_size, 0644 | IPC_CREAT);

        // Membuat shared memory untuk semaphore
    sem_size = sizeof(sem_t);
    semid = shmget(key2, sem_size, 0644 | IPC_CREAT);

        // Menghubungkan shared memory ke program
    ptr = (int *) shmat(shmid, (void *)0, 0);
    (*ptr) = 0;

        // Menghubungkan Semaphore ke shared memory
    mutex = (sem_t *) shmat(semid, (void *)0, 0);
    sem_init(mutex, 1, 1);   // initialise semaphore

    if (fork() == 0) {   // child process
        for (i = 0; i < nloop; i++) {
```

```
            value = ++(*ptr);
            printf("child process %d %d: %d\n", getpid(), get
ppid(), value);
            fflush(NULL);
        }
        exit(0);
    }

    // parent process
    for (i = 0; i < nloop; i++) {
        value = ++(*ptr);
        printf("parent process %d %d: %d\n", getpid(), getppi
d(), value);
        fflush(NULL);
    }
        // Parent dan child akan berjalan bersamaan dan sama-
sama mengakses shared memory *ptr

        // Parent menunggu child
    wait(&status);

        // Melepaskan shared memory
    shmdt(ptr);
    shmdt(mutex);

        // Shared memory dihapus dari sistem
    shmctl(shmid, IPC_RMID, (struct shmid_ds *)0);
    shmctl(semid, IPC_RMID, (struct shmid_ds *)0);

    exit(0);
}
```

HASILNYA:

```
child process 7134 7133: 999129
child process 7134 7133: 999130
child process 7134 7133: 999131
child process 7134 7133: 999132
child process 7134 7133: 999133
child process 7134 7133: 999134
child process 7134 7133: 999135
child process 7134 7133: 999136
child process 7134 7133: 999137
child process 7134 7133: 999138
child process 7134 7133: 999139
child process 7134 7133: 999140
child process 7134 7133: 999141
child process 7134 7133: 999142
child process 7134 7133: 999143
child process 7134 7133: 999144
child process 7134 7133: 999145
child process 7134 7133: 999146
child process 7134 7133: 999147
child process 7134 7133: 999148
child process 7134 7133: 999149
child process 7134 7133: 999150
child process 7134 7133: 999151
vboxuser@Ubuntu:~/Desktop/week05$
```

Kenapa bisa child semua? Karena OS Scheduler

1. How many processes are created? What variables and resources are accessed simultaneously byeach process? What instructions are in the critical section? What are the possible outcomes?

Program ini menjalankan fungsi `fork()` akan menduplikasi process yang sedang berjalan akibatnya akan terbentuk Parent Process dan Child Process. Jadi jumlah process ada 2 process.

Kedua process mengakses shared memory yang sama dimana:

```
// Shared Memory dibuat dengan
shmid = shmget(key, shm_size, 0644 | IPC_CREAT);

// Kemudian dihubungkan dengan
ptr = (int *) shmat(shmid, (void *)0, 0);

// Variabel yang di-shared adalah:
*ptr // Ini adalah integer dalam shared memory yang digunakan
sebagai counter

// Sealin itu kedua process mengakses
mutex
```

Critical Section / Critical Region adalah bagian program yang mengakses shared resource dan tidak boleh dijalankan bersamaan. Pada program tersebut critical sectionnya:

```
value = ++(*ptr);
// Hal ini karena:
// Parent process menambah nilai
// Child process juga menambah nilai
// Jika kedua process ini dijalankan secara bersamaan maka ak
an terjadi race condiiton
```

Possible Outcomes yang dapat terjadi tampa sinkronisasi adalah output yang random / non-deterministik. Jika ada sinkronisasi maka hasilnya akan tetap yaitu `500000 + 500000 = 1000000`

2. Also note the differences in the arguments for sem_init() compared to those used in praksem-2.c. Explain the second argument for sem_init()

Perbedaannya pada praksem-2.c yaitu:

```
// pada praksem2.c
sem_init(&semid1, 0, 0)
```

```
// pada praksem3.c
sem_init(mutex, 1, 1);

// ARGUMEN KEDUA DIGUNAKAN UNTUK:
/*
- Jika nilainya 0, maka semaphore digunakan antar thread
- Jika nilainya 1, maka semaphgore digunakan antar process
*/

// ARGUMEN KETIGA DIGUNAKAN UNTUK: Inisialisasi Nilai awal se
maphore
```

Perbedaannya hanya ada parameter dimana:

Parameter	praksem-2	praksem-3
pshared	0	1
value	0	1

3. Does a race condition Occur ?

Ya, walaupun semaphore dibuat, tetapi tidak digunakan pada critical section dan tidak ada operasi:

```
sem_wait()
sem_post()
```

Akibatnya kedua process bisa mengakses `++(*ptr)` secara bersamaan, hal ini menyebabkan race condition

4. Revise the program by using a binary semaphore to achieve mutual exclusion at the critical region Place a pair of sem_wait() and sem_post operations at the appropriate positions to synchronize
access to shared variables/resources.

```
// SAVE DENGAN NAMA : praksem-3_NIM.c
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <semaphore.h>

#define key  ((key_t) 7890)
#define key2 ((key_t) 7891)

int main(int argc, char **argv)
{
    int i, nloop = 500000, value, *ptr;
    int shmid, semid, shm_size, sem_size, status;
    sem_t *mutex;

    /* membuat shared memory untuk integer */
    shm_size = sizeof(int);
    shmid = shmget(key, shm_size, 0644 | IPC_CREAT);

    /* membuat shared memory untuk semaphore */
    sem_size = sizeof(sem_t);
    semid = shmget(key2, sem_size, 0644 | IPC_CREAT);

    /* attach shared memory */
    ptr = (int *) shmat(shmid, (void *)0, 0);
    (*ptr) = 0;

    /* attach semaphore */
    mutex = (sem_t *) shmat(semid, (void *)0, 0);

    /* inisialisasi binary semaphore */
    sem_init(mutex, 1, 1);
```

```
    if (fork() == 0) {   // child process

        for (i = 0; i < nloop; i++) {

            sem_wait(mutex);          // masuk critical secti
on

            value = ++(*ptr);

            sem_post(mutex);          // keluar critical sect
ion

            printf("child process %d %d: %d\n",
                   getpid(), getppid(), value);
            fflush(NULL);
        }

        exit(0);
    }

    /* parent process */
    for (i = 0; i < nloop; i++) {

        sem_wait(mutex);              // masuk critical secti
on

        value = ++(*ptr);

        sem_post(mutex);              // keluar critical sect
ion

        printf("parent process %d %d: %d\n",
               getpid(), getppid(), value);
        fflush(NULL);
    }
```

```
    wait(&status);

    /* detach shared memory */
    shmdt(ptr);
    shmdt(mutex);

    /* hapus shared memory */
    shmctl(shmid, IPC_RMID, (struct shmid_ds *)0);
    shmctl(semid, IPC_RMID, (struct shmid_ds *)0);

    exit(0);
}
```

HASIL :

```
child process 7258 7257: 999978
child process 7258 7257: 999979
child process 7258 7257: 999980
child process 7258 7257: 999981
child process 7258 7257: 999982
child process 7258 7257: 999983
child process 7258 7257: 999984
child process 7258 7257: 999985
child process 7258 7257: 999986
child process 7258 7257: 999987
child process 7258 7257: 999988
child process 7258 7257: 999989
child process 7258 7257: 999990
child process 7258 7257: 999991
child process 7258 7257: 999992
child process 7258 7257: 999993
child process 7258 7257: 999994
child process 7258 7257: 999995
child process 7258 7257: 999996
child process 7258 7257: 999997
child process 7258 7257: 999998
child process 7258 7257: 999999
child process 7258 7257: 1000000
vboxuser@Ubuntu:~/Desktop/week05$
```

DETERMINISTIC OUTPUT :

```
parent process 7273 2856: 999978
parent process 7273 2856: 999979
parent process 7273 2856: 999980
parent process 7273 2856: 999981
parent process 7273 2856: 999982
parent process 7273 2856: 999983
parent process 7273 2856: 999984
parent process 7273 2856: 999985
parent process 7273 2856: 999986
parent process 7273 2856: 999987
parent process 7273 2856: 999988
parent process 7273 2856: 999989
parent process 7273 2856: 999990
parent process 7273 2856: 999991
parent process 7273 2856: 999992
parent process 7273 2856: 999993
parent process 7273 2856: 999994
parent process 7273 2856: 999995
parent process 7273 2856: 999996
parent process 7273 2856: 999997
parent process 7273 2856: 999998
parent process 7273 2856: 999999
parent process 7273 2856: 1000000
vboxuser@Ubuntu:~/Desktop/week05$
```

2. PRACTICUM 5.2 : Named Semaphore

Named Semaphore adalah semaphore yang diidentifikasi dengan sebuah nama string dan disimpan pada file system. Anda dapat mengeceknya pada direktori `/dev/shm` dengan format nama `sem.nama_semaphore` , Keberadaan named semaphore memungkinakn process yang tidak saling berhubungan untuk melakukan sinkronisasi saat mengakses variabel atau resource bersama.

Named semaphore dibuka menggunakan fungsi:

```
sem_open()
```

dan semaphore tersebut akan tetap ada sampai secara khusus dihapus menggunakan:

```
sem_unlink()
```

ANALOGI PERBEDAAN Unnamed dan Named Semaphore : Kunci Ruang Meeting

- Unnamed Semaphore, Kunci hanya ada didalam satu ruangan, hanya orang di ruangan itu yang bisa memakainya.
- Named Semaphore, Kunci disimpan di meja resepsionis dengan label nama. Semua orang dari department mana pun bisa meminjam kunci itu.

Kedua program dibawah ini menggunakan shared memory dan named semaphore untuk melakukan sinkronisasi antar process yang berbeda

Program ini terdiri dari dua proses berbeda:

Program	Peran
server.c	menulis data ke shared memory
client.c	membaca data dari shared memory

Keduanya menggunakan:

- **shared memory** → tempat menyimpan data bersama
- **named semaphore (`mysem`)** → mengatur akses ke shared memory

1. server_NIM.c

```
// SAVE DENGAN NAMA server_NIM.c
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```c
#include <stdio.h>
#include <semaphore.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

#define SHMSZ 27
char SEM_NAME[] = "mysem";

int main()
{
    char ch;
    int shmid;
    key_t key;
    char *shm, *s;
    sem_t *mutex;

    // name the shared memory segment
    key = 1000;

    // create & initialize semaphore
    mutex = sem_open(SEM_NAME, O_CREAT, 0644, 1);

    // create the shared memory segment with this key
    shmid = shmget(key, SHMSZ, IPC_CREAT | 0666);

    /* attach this segment to shared memory */
    shm = shmat(shmid, NULL, 0);

    s = shm;

    for (ch = 'A'; ch <= 'Z'; ch++) {
        sem_wait(mutex);
        *s++ = ch;
        sem_post(mutex);
    }
```

```
    // the below loop could be replaced by binary semaphore
    while (*shm != '*') {
        sleep(1);
    }

    sem_close(mutex);
    sem_unlink(SEM_NAME);
    shmctl(shmid, IPC_RMID, 0);

    exit(0);
}
```

2. client_NIM.c

```
// SAVE DENGAN NAMA : client_NIM.c
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <semaphore.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

#define SHMSZ 27
char SEM_NAME[] = "mysem";

int main()
{
    char ch;
    int shmid;
    key_t key;
```

```
    char *shm, *s;
    sem_t *mutex;

    // name the shared memory segment
    key = 1000;

    // create & initialize semaphore
    mutex = sem_open(SEM_NAME, O_CREAT, 0644, 1);

    // create the shared memory segment with this key
    shmid = shmget(key, SHMSZ, IPC_CREAT | 0666);

    // attach this segment to shared memory
    shm = shmat(shmid, NULL, 0);

    // the below loop could be replaced by binary semaphore
    while (*shm != 'A') {
        sleep(1);
    }

    for (s = shm; *s != '\0'; s++) {
        sem_wait(mutex);
        putchar(*s);
        sem_post(mutex);
    }

    printf("\n");

    // once done signal exiting (this can be replaced by anot
her semaphore)
    *shm = '*';

    sem_close(mutex);
    sem_unlink(SEM_NAME);
    shmctl(shmid, IPC_RMID, 0);
```

```
    exit(0);
}
```

3. Penjelasan:

1. Compile and run the two programs above on two different terminals. Explain what these two programs do.

Cara Menjalankan Program:

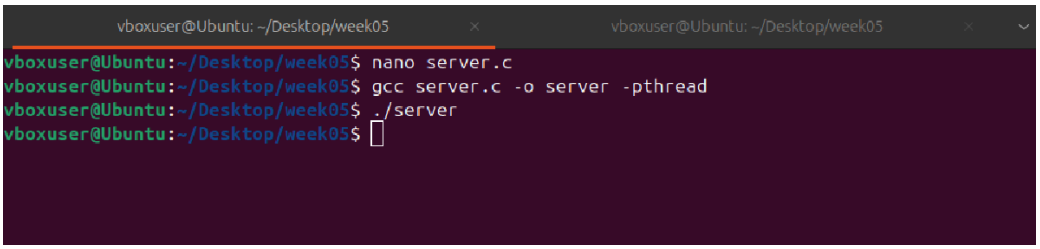
```
// Ada dua program berbeda
server_NIM.c
client_NIM.c

// Compile
gcc server_NIM.c -o server
gcc client_NIM.c -o client

// Jalankan pada terminal berbeda
```

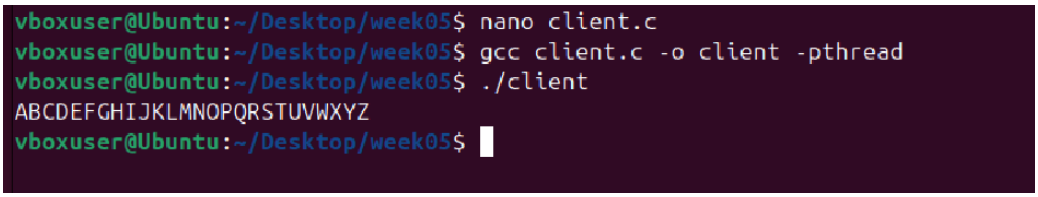
Tujuan dari kedua program ini untuk menunjukkan komunikasi antar process menggunakan shared memory dan named semaphore.

Alur kerjanya:

- Server membuat semaphore dan shared memory
- Server menulis huruf A hingga Z ke shared memory
- Client menunggu sampai huruf A muncul
- Client membaca data dari shared memory
- Client mencetak A hingga Z ke layar
- Client menuliskan tanda * ke shared memory
- Server mendeteksi tanda * dan berhenti
- Shared memory dan semaphore dihapus

HASILNYA:





2. Explain the purpose of using a semaphore variable mutex in this program!

Tujuan dari penggunaan Semaphore Mutex

Variabel:

```
sem_t*mutex;
```

digunakan sebagai **semaphore untuk mutual exclusion**.

Artinya semaphore ini berfungsi untuk mengontrol akses ke shared memory agar server dan client tidak mengakses data secara bersamaan. HI ini penting karena jika kedua process melakukan operasi ini pada waktu yang sama, maka dapat terjadi Race Condition dan Outputnya Non-Deterministik.

Cara kerjanya :

Server:

```
sem_wait(mutex);
*s++=ch;
sem_post(mutex);
```

Client:

```
sem_wait(mutex);
putchar(*s);
```

```
sem_post(mutex);
```

Alurnya:

```
sem_wait → mengunci resource
akses shared memory
sem_post → membuka resource
```

Sehingga hanya **satu proses yang dapat mengakses shared memory pada satu waktu**.

3. How does the server wait for the client?

Server menunggu client dengan menggunakan **polling loop** berikut:

```
while (*shm!='*')
{
    sleep(1);
}
```

Server akan terus memeriksa isi dari shared memory, Selama nilai pertama shared memory bukan *, maka server akan menunggu dan sleep 1 detik dan akan melakukan pengecekan ulang.

4. Modify the two programs above so that there is synchronization where the server exits after there is information from the client which is done using semaphore.

```
// SAVE DENGAN NAMA : serverMOD_NIM.c
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
```

```
#include <semaphore.h>
#include <sys/stat.h>
#include <fcntl.h>

#define SHMSZ 27

char MUTEX_NAME[] = "mysem";
char SYNC_NAME[]  = "syncsem";

int main()
{
    char ch;
    int shmid;
    key_t key;
    char *shm, *s;

    sem_t *mutex;
    sem_t *sync;

    key = 1000;

    /* membuka semaphore */
    mutex = sem_open(MUTEX_NAME, O_CREAT, 0644, 1);
    sync  = sem_open(SYNC_NAME, O_CREAT, 0644, 0);

    /* membuat shared memory */
    shmid = shmget(key, SHMSZ, IPC_CREAT | 0666);

    /* attach shared memory */
    shm = shmat(shmid, NULL, 0);
    s = shm;

    /* server menulis huruf A-Z */
    for (ch = 'A'; ch <= 'Z'; ch++) {

        sem_wait(mutex);
```

```
        *s++ = ch;

        sem_post(mutex);
    }

    *s = '\0';

    printf("Server finished writing. Waiting for clien
t...\n");

    /* menunggu sinyal dari client */
    sem_wait(sync);

    printf("Client finished reading. Server exiting...\n");

    sem_close(mutex);
    sem_close(sync);

    sem_unlink(MUTEX_NAME);
    sem_unlink(SYNC_NAME);

    shmctl(shmid, IPC_RMID, 0);

    exit(0);
}
```

```
// SAVE DENGAN NAMA : clientMOD_NIM.c
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <semaphore.h>
```

```
#include <sys/stat.h>
#include <fcntl.h>

#define SHMSZ 27

char MUTEX_NAME[] = "mysem";
char SYNC_NAME[]  = "syncsem";

int main()
{
    int shmid;
    key_t key;
    char *shm, *s;

    sem_t *mutex;
    sem_t *sync;

    key = 1000;

    /* membuka semaphore */
    mutex = sem_open(MUTEX_NAME, O_CREAT, 0644, 1);
    sync  = sem_open(SYNC_NAME, O_CREAT, 0644, 0);

    /* mendapatkan shared memory */
    shmid = shmget(key, SHMSZ, IPC_CREAT | 0666);

    /* attach shared memory */
    shm = shmat(shmid, NULL, 0);

    printf("Client reading data:\n");

    for (s = shm; *s != '\0'; s++) {

        sem_wait(mutex);

        putchar(*s);
```

```
        sem_post(mutex);
    }

    printf("\nClient finished reading\n");

    /* memberi sinyal ke server */
    sem_post(sync);

    sem_close(mutex);
    sem_close(sync);

    exit(0);
}
```

Kalau yang metode lama menggunakan `while ( shm != ' ')` yang akan menimbulkan busy waiting dan CPU tetap terjaga. Sedangkan pada metode baru menggunakan `sem_wait(sync)` sleep sampai ada sinyal dan lebih efisien dan sinkronisasi lebih baik.

C. PRACTICUM 5.3: Pthread Mutex

Buka program multithread dari Modul3 yang menunjukkan race condition. Identifikasi critical region pada program tersebut. Gunakan mutex dari pthread library untuk mmebuat kondisi Mutual Exclusion.

Include a global mutex variable with the declaration pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

> Lock the critical region of the program by adding the following instructions:

```
pthread_mutex_lock(&mutex);
..... Critical Region ........
pthread_mutex_unlock(&mutex);
```

PROGRAM WEEK 3 :

```
// SAVE dengan nama prak8-4_NIM.c

// Program ini dibuat untuk mendemonstraksikan:
// Race Condition akibat operasi increment pada shared variab
el tanpa sinkronisasi
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

#define NITERS 10000000

void count(void *arg);
unsigned int cnt = 0;

int main() {
pthread_t tid1, tid2;

// Kedua thread dibuat dan mengakses fungsi count masing-masi
ng
pthread_create(&tid1, NULL, (void *)count, NULL);
pthread_create(&tid2, NULL, (void )count, NULL);

/
Hal yang terjadi di CPU :
- Load cnt dari memory
- Tambah 1
- Store kembali ke memory
- Secara matematis harusnya x *  NITTERS, cnt = 20000000
- Karena tidak adanya sinkronisasi, pasti akan menimpa satu s
ama lain
- Sehingga nilai NITTERS < 20000000
/
```

```
pthread_join(tid1, NULL);
pthread_join(tid2, NULL);

if(cnt != (unsigned)NITERS2) {
printf("Wrong... cnt=%d\n", cnt);
} else {
printf("OK...\tcnt=%d\n", cnt);
}
exit(0);
}

void count(void *arg) {
int i;
for(i = 0; i < NITERS; i++) {
cnt++;
}
}// SAVE DENGAN NAMA : prak3_NIM.c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <stdio.h>

// Shared Global Variabel
int g1 = 0;

void task1(int *counter) {
    while(*counter < 10) {
        printf("task1 count : %d\n", *counter);
        (*counter)++;
        sleep(1);
    }
    g1 = 10;
}

/*
task1 :
```

```
- Loop 10 kali
- Cetak nilai Counter
- Increment
- Delay 1 detik
- Setelah selesai set g1 = 10
*/

void task2(int *counter) {
    while(*counter < 10) {
        printf("task2 count : %d\n", *counter);
        (*counter)++;
        sleep(1);
    }
    g1 = 20;
}

/*
task2 :
- Loop 10 kali
- Cetak nilai Counter
- Increment
- Delay 1 detik
- Setelah selesai set g1 = 20

*/

// STEP 1 : Main thread dimulai
int main() {
    pthread_t thrd1, thrd2;
    int a = 0, b = 0, ret;

    printf("Global variable g1 = %d\n", g1);

    // Membuat thread1, dan diberi alamat variabel a, dimana
a sekarang = 0
    ret = pthread_create(&thrd1, NULL, (void*)task1, (void*)&
```

```
a);
    if(ret != 0) {
        perror("pthread_create: task1\n");
        exit(EXIT_FAILURE);
    }

    ret = pthread_create(&thrd2, NULL, (void*)task2, (void*)&
b);
    if(ret != 0) {
        perror("pthread_create: task2\n");
        exit(EXIT_FAILURE);
    }

    // Kedua thread berjalan berbarengan
    // Interleaving Execution
    pthread_join(thrd2, NULL);
    pthread_join(thrd1, NULL);
    // Siapa yang selesai terakhir yang akan menentukan nilai
g1
    // misal task2 selesai duluan, maka nilai g1 adalah 20
    printf("Global variabel g1 = %d\n", g1);

}
```

1. Penjelasan:

a. Identifikasi Critical Region pada Program tersebut!

Pada program tersebut, Critical Regionnya adalah

```
cnt++;

// Mengapa ini merupakan critical regionnya?
// Karena operasi cnt++ sebenarnya terdiri dari 3 operasi CPU
/*
```

```
1. Load cnt dari memory
2. Tambah 1
3. Store kembali ke memory

Jika 2 thread menjalankan ini secara bersamaan maka akan terj
adi INTERLEAVING
*/
```

b. Modifikasi Program !

```
// SAVE dengan nama prak8-4_NIM.c

#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

#define NITERS 10000000

void *count(void *arg);

unsigned int cnt = 0;

/* GLOBAL MUTEX */
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int main() {

    pthread_t tid1, tid2;

    pthread_create(&tid1, NULL, count, (void*)"Thread 1");
    pthread_create(&tid2, NULL, count, (void*)"Thread 2");

    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
```

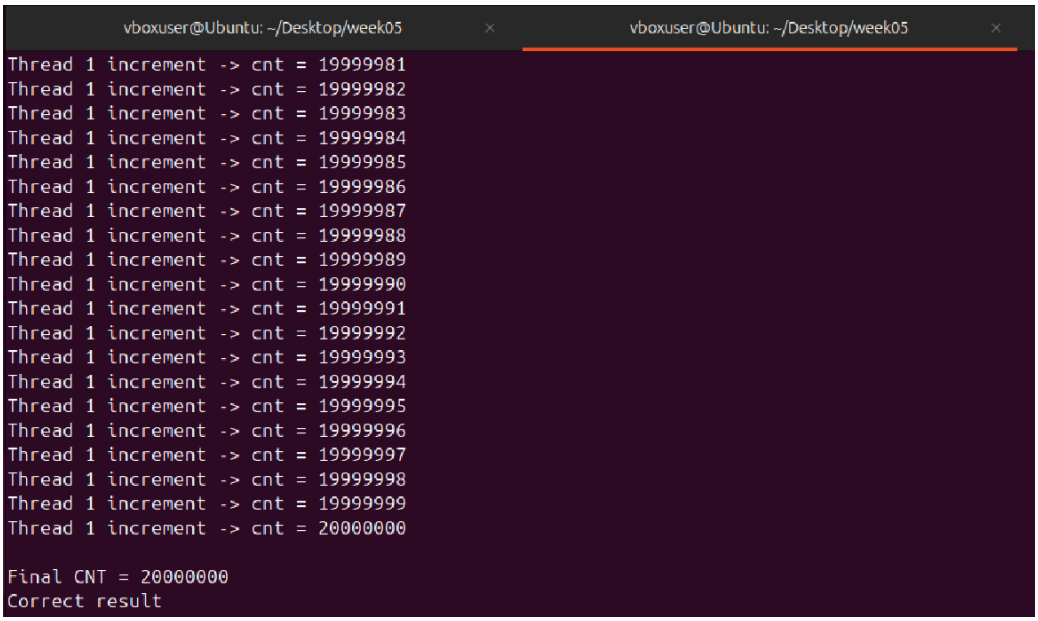
```
    printf("\nFinal CNT = %u\n", cnt);

    if(cnt != (unsigned)NITERS*2) {
        printf("Wrong result\n");
    } else {
        printf("Correct result\n");
    }

    return 0;
}

void *count(void *arg) {

    char *thread_name = (char*)arg;

    for(int i = 0; i < NITERS; i++) {

        pthread_mutex_lock(&mutex);

        cnt++;

        printf("%s increment -> cnt = %u\n", thread_name, cn
t);

        pthread_mutex_unlock(&mutex);
                // Microsecond
        // usleep(1); // delay supaya interleaving terlihat
    }

    return NULL;
}
```

Degan adanya Mutex, alur program yang terjadi akan menjadi seperti:

- Thread1 lock mutex
- Thread1 increment

- Thread1 unlock mutex
- Thread2 baru bisa increment

HASILNYA :



2. SOAL : "Explain some of the differences between the use of semaphores and pthread mutexes." Jelaskan dengan kata-kata sendiri!

SAVE DENGAN NAMA : jawaban_NIM.txt